

SOLEN

Solen Architecture and Implementation Guide

A modular settlement network with native rollups, smart accounts, privacy primitives, and intent-aware execution.

A build-oriented specification for the protocol, execution environments, client software, operations, and launch process.

Version	Version 0.1 - March 29, 2026
Status	Design proposal for implementation
Audience	Protocol engineers, client developers, application teams, security reviewers
Scope	Base layer, rollups, accounts, privacy, governance, node software, SDKs, operations

Version 0.1 - March 29, 2026

Contents

1. Product Definition	3
2. Reference Architecture	3
3. Functional Requirements	4
4. Non-functional Requirements	5
5. Protocol Design Decisions	6
6. Software Components and Deliverables	7
7. Recommended Repository Structure	7
8. Data Model and State Layout	8
9. API Surface	8
10. Security Program	9
11. Developer Experience Plan	9
12. Phased Delivery Plan	10
13. Mainnet Readiness Checklist	10
14. Open Research and Deferred Decisions	11
15. Suggested Immediate Next Steps	11

1. Product Definition

Solen is a clean-room redesign of a general-purpose blockchain ecosystem. The base layer is intentionally narrow: consensus, settlement, asset accounting, data publication, canonical bridge logic, proof verification, governance hooks, and a minimal execution environment for system contracts. Most user-facing application execution occurs on native rollups or specialized execution domains that settle to Solen L1.

Primary design goals

Keep the base layer simple, make rollups first-class, eliminate externally owned accounts, reduce MEV harm, support bounded privacy, and hide protocol complexity behind modern wallet abstractions.

The final product consists of six tightly integrated surfaces:

- Solen L1 protocol and validator client software.
- Canonical rollup framework with proof, bridge, sequencing, and messaging interfaces.
- Smart-account standard library and wallet SDK.
- Data-availability and state-witness services for nodes, provers, and indexers.
- Developer tooling: compiler, local devnet, RPC stack, explorers, SDKs, and contract templates.
- Operational stack: monitoring, key management, incident response, upgrade pipelines, and launch governance.

2. Reference Architecture

A programmer building Solen should treat the system as a collection of versioned subsystems with strict interfaces rather than one giant codebase. The implementation should be split into repositories or modules that can be audited, benchmarked, and upgraded independently.

Subsystem	Purpose	Primary outputs
Consensus	Validator networking, block production, finality, slashing, checkpointing.	Finalized L1 blocks and validator attestations
Settlement kernel	Maintains assets, system contracts, canonical bridge, proof registry, fee accounting.	State roots, bridge balances, proof receipts
Rollup framework	Defines batch publication, fraud/validity proof	Rollup commitments and message receipts

Subsystem	Purpose	Primary outputs
	verification, messaging, and sequencer policy.	
Execution domains	Native rollups or app-specific environments.	Transactions, state transitions, domain proofs
Account stack	Smart accounts, session keys, recovery, sponsorship, permissioning.	User operations and signatures
Node services	RPC, indexing, witness generation, archival, explorer APIs.	Queryable state and developer interfaces

3. Functional Requirements

3.1 Base layer requirements

- Deterministic block execution and finalization under a Byzantine fault-tolerant proof-of-stake design.
- Canonical representation of native assets, bridged assets, sequencer bonds, validator stakes, and fee sinks.
- Native support for rollup batch publication and proof verification.
- A system-contract environment for governance, bridge logic, proof verifiers, stake management, and treasury functions.
- Explicit event emission for all state transitions that off-chain components must index.

3.2 Rollup requirements

- A rollup registration process with parameter schemas, proof type selection, chain identifiers, and governance-controlled admission.
- Canonical ingress and egress bridge contracts shared across rollups.
- Cross-rollup messaging with receipts, replay protection, timeout semantics, and fee payment policy.
- Support for multiple execution VMs provided they implement standard state commitment and proof interfaces.
- Sequencer policy that allows centralized bootstrapping but upgrades toward decentralized sequencing or shared sequencing.

3.3 Account requirements

- All user accounts are programmable smart accounts, never EOAs.
- Authentication methods include passkeys, hardware keys, threshold recovery, guardians, and session credentials.
- Gas abstraction permits fee sponsorship and payment in approved assets through paymaster contracts.
- Wallets must support batched operations, spending policies, revocation, and human-readable transaction simulation.

3.4 Privacy requirements

- Selective-disclosure attestations for claims such as membership, age, jurisdiction, or KYC completion.
- Shielded transfer support at least on selected execution domains, even if L1 remains mostly transparent.
- Zero-knowledge proof interfaces for private balances, private voting, or privacy-preserving compliance proofs.

4. Non-functional Requirements

Area	Target direction
Security	Formal specs for critical contracts; third-party audits; fuzzing; differential testing; hardware key support; key isolation for validators and sequencers.
Performance	Predictable block times, bounded state growth, batch-friendly proof verification, and horizontal scaling in non-consensus services.
Reliability	Crash-safe state persistence, node restart integrity, deterministic replay, blue-green deployments for RPC/indexing services.
Operability	Metrics, traces, structured logs, health checks, checkpoint restore, disaster recovery runbooks, and upgrade automation.
Developer experience	One-command devnet, clear error surfaces, reference wallets, exhaustive JSON-RPC docs,

Area	Target direction
	versioned SDKs, and test templates.

5. Protocol Design Decisions

5.1 Consensus

Choose a proof-of-stake protocol with deterministic finality, slashing, and efficient light-client proofs. The implementation should separate networking, fork choice, block execution, and validator key management. Build a formal state machine for epoch transitions, rewards, penalties, and emergency recovery. Avoid making the consensus engine aware of application-specific semantics beyond system contracts and fee accounting.

5.2 Settlement kernel

Implement the kernel as minimal native modules or highly constrained system contracts. Keep balance accounting, stake accounting, proof registry, bridge vaults, treasury, and governance isolated. Every state transition should emit structured events that can be indexed consistently across languages.

5.3 Virtual machine strategy

Do not use the historical EVM as the primary long-term VM. Start with a modern deterministic VM or a WASM-like environment, plus strict gas/resource metering. Provide a compiler toolchain that targets the VM from memory-safe languages. System contracts can use a restricted runtime distinct from application domains.

5.4 Fee market and intents

Internally preserve metered resource pricing, but expose user intent APIs rather than forcing users to manually reason about gas. A wallet submits a signed intent or user operation; bundlers or paymasters transform it into protocol transactions. Add explicit simulation results, max spend controls, and failure reasons.

5.5 MEV controls

Introduce encrypted mempools, batch auctions, or intent matching at the protocol edge. Start simple, but build interfaces that let the protocol capture and redistribute ordering value rather than leaking it entirely to private actors. Preserve auditable ordering rules.

6. Software Components and Deliverables

Component	Must ship	Notes
Validator client	Consensus engine, P2P stack, execution interface, slashing protection, key management hooks.	Must support reproducible builds and checkpoint sync.
Execution client	Settlement execution, state storage, system contracts, proof verification APIs.	Should expose deterministic replay and debug tracing.
Rollup node kit	Sequencer, batch publisher, prover adapter, cross-domain messenger, bridge relayer.	Keep pluggable proof systems and DA backends.
Wallet SDK	Smart account deployment, passkey auth, sponsorship, recovery, policy engine.	Reference implementations for web, mobile, and server use.
RPC/indexer	Public JSON-RPC, event indexing, explorer API, receipts, historical queries.	Separate hot path from archival path.
Dev tooling	Compiler, local devnet, faucet, testing harness, templates, docs site, package registry.	Aim for a one-hour first-app experience.

7. Recommended Repository Structure

Suggested top-level repositories

solen-consensus, solen-execution, solen-system-contracts, solen-rollup-kit, solen-wallet-sdk, solen-rpc, solen-explorer, solen-devnet, solen-specs, solen-audits

- Keep protocol specifications separate from implementation repositories so multiple clients can be built against the same spec.
- Treat generated code, ABI files, protobuf/IDL files, and test vectors as versioned build artifacts.

- Pin serialization rules early and create compatibility tests that all clients must pass.

8. Data Model and State Layout

Define state explicitly before coding. Solen should publish a canonical state schema for balances, stakes, validator sets, registered rollups, bridge vaults, message queues, nonces, account policies, proof receipts, and governance objects. The schema should be implementation-agnostic and suitable for test-vector generation.

Object	Key fields
Account	account_id, code_hash, owners, threshold, guardians, nonce lanes, spending policies, session keys
Validator	validator_id, pubkeys, stake, status, slashing history, reward destination
Rollup	rollup_id, VM type, proof type, DA mode, bridge config, sequencer set, governance params
Bridge vault	asset_id, origin_domain, supply, custody state, pending exits, challenge windows
Message receipt	source, destination, nonce, payload hash, timeout, proof reference, execution status

9. API Surface

A high-quality implementation guide must specify the external APIs up front. Publish JSON-RPC and gRPC schemas for node queries, transaction submission, intent submission, simulation, event subscriptions, proof publication, bridge status, and account recovery flows. Every endpoint should include authentication assumptions, pagination, error codes, and consistency guarantees.

- State queries: latest state, historical state, proofs, receipts, events, bridge balances, validator status.
- Write APIs: submit transaction, submit user operation, submit intent, submit batch, publish proof, dispute proof.
- Simulation APIs: dry-run, gas estimate, policy check, sponsorship check, decoding and human-readable action summary.

- Operational APIs: peer health, sync status, metrics, backups, checkpoints, and admin-safe maintenance commands.

10. Security Program

1. Write executable specifications and adversarial test cases for all critical system contracts and consensus transitions.
2. Stand up fuzzing infrastructure for transaction decoding, networking, VM execution, bridge flows, and cross-domain messaging.
3. Run differential tests across reference and optimized clients.
4. Commission independent audits for consensus, execution, wallet/account abstraction, bridge, prover/verifier logic, and cryptographic libraries.
5. Adopt a staged bug bounty: private pre-launch, limited public beta, then broad public program after the first stable release.

11. Developer Experience Plan

A chain without good tooling will fail even if the protocol is strong. Solen should budget substantial effort for developer ergonomics in the first release, not after launch.

Track	Required deliverable by v1
Documentation	Architecture overview, API references, tutorials, node operator handbook, wallet integration guide, rollup developer guide.
Testing	Local devnet, snapshot fixtures, fork tests, property-based tests, contract templates, CI recipes.
SDKs	TypeScript and Rust first; mobile support shortly after; examples for account abstraction and bridge flows.
Exploration	Explorer, log/event viewer, transaction simulator, state diff viewer, and proof inspection tools.

12. Phased Delivery Plan

Phase	Goal	Exit criteria
Phase 0 - Specification	Freeze architectural decisions, state schemas, interface definitions, and threat model.	Specs versioned; reference tests defined; core cryptographic choices locked.
Phase 1 - Single-node prototype	Demonstrate deterministic execution, accounts, bridge skeleton, and RPC basics.	Reproducible local devnet with scripted test flows.
Phase 2 - Multi-node alpha	Consensus, staking, event indexing, rollup registration, batch publication.	Stable multi-validator testnet; checkpoint sync works.
Phase 3 - Security hardening	Audits, fuzzing, rollback drills, metrics, backups, wallet UX, governance controls.	Critical findings resolved; incident playbooks tested.
Phase 4 - Public testnet	Open developer access, external rollups, incentives for breakage discovery.	SLOs achieved; bug bounty active; docs complete.
Phase 5 - Mainnet launch	Limited initial validator and rollup set with conservative parameter choices.	Launch checklist signed off; emergency controls in place.

13. Mainnet Readiness Checklist

- Genesis ceremony completed with independently verified artifacts.
- Validator binaries reproduced from source by at least two independent teams.
- Checkpoint restore, snapshot recovery, and rollback drills executed successfully.
- Bridge limits, withdrawal delays, and sequencer bonds configured conservatively.
- Wallet simulation, sponsorship, and recovery flows validated on production-like environments.
- Monitoring dashboards, on-call rotations, incident severity matrix, and communications templates prepared.
- Legal and policy review completed for token distribution, foundation operations, grants, and sanctions handling where applicable.

14. Open Research and Deferred Decisions

The guide should acknowledge what is not fully solved yet. Document unresolved design areas rather than hiding them in code comments.

- Final proof system selection and verifier footprint on L1.
- Whether shared sequencing is protocol-native or marketplace-based in early releases.
- The privacy model for L1 versus privacy-focused execution domains.
- State rent or state expiry policy, and how archival history is funded.
- Governance minimization strategy after initial bootstrap control is no longer needed.

15. Suggested Immediate Next Steps

6. Write the Solen specification repository first: architecture spec, terminology, state schema, RPC schema, and test vectors.
7. Prototype the account abstraction stack and wallet simulation experience before finalizing application VM details.
8. Build a thin execution kernel plus one canonical rollup demo to validate bridge and messaging assumptions.
9. Create a local devnet that launches L1, one rollup, one prover adapter, and one wallet demo in a single command.
10. Begin security reviews early, especially around cross-domain message replay, bridge accounting, and sponsorship policy abuse.